Master Checklist — SecretHandle (Full Research Paper)

Kế hoạch toàn diện từ thiết kế hệ thống đến thực nghiệm empirical A→Z

Cập nhật: 2026-10-29 | Mục tiêu: **Full Research Paper** (ACM MobiSys / USENIX Security / ACM CCS / NDSS / IEEE S&P) | Nguyên tắc: không placeholder nào được trình bày như kết quả.

Ký hiệu trạng thái: ■ Đã xong ⊡ Đang làm / một phần □ Chưa bắt đầu ▶ Đang tắc (blocker)

Mốc quan trọng (Lộ trình Full Research Paper)

Cổng	Giai đoạn	Mục tiêu đạt được (Full Paper)
G0	Tuần 1–2	Môi trường dev sẵn sàng; **B0 tái hiện được** trên emulator/stock; chốt D0–D6 **(ĐẠT 2026-10-17)**
G1	Tuần 3–5	Tái hiện đầy đủ họ tấn công (1, 3, 4, 7) + mô hình tham chiếu Python 6.400 trials + khung thiết kế hệ thống **(ĐẠT 2026-10-17; `docs/gates/G1.md`)**
G2	Tuần 6–8	**Bộ khung thực nghiệm trên EC2**: Đo Attack 1 ($n = 30$ B0 vs B3 ■); Attack 2 (Destination Redirection); Baseline B_1; Attack 4 (Replay & Expiry); Microbenchmarks độ trễ ms (RQ5)
G3	Tuần 9–12	**Đa dạng hóa ứng dụng & Benchmark**: Workloads A–D, AndroidWorld benign regression, đa dạng hóa LLM (Claude Sonnet/Haiku, GPT-4o)
Camera	Tuần 13–16	Đóng băng số liệu thực nghiệm, hoàn thiện bản thảo Full Paper (14 trang format hội nghị) + đóng gói Artifact mã nguồn mở

PHẦN 0 — Tiêu chuẩn & Phạm vi của Full Research Paper

Một bài Full Research Paper tại các hội nghị bảo mật/hệ thống hàng đầu đòi hỏi tính hoàn thiện cao cả về lý thuyết, cài đặt hệ thống và thực nghiệm empirical đối đầu:

- ■ **Vấn đề & tác động thực tế**: Rò rỉ OTP và bí mật cho tác nhân AI di động (đã tái hiện thực tế B_0 trên Android Emulator, $29/30$ rò rỉ, Wilson $[0.833, 0.994]$).
- ■ **Mô hình đe dọa & caller principal**: 3 kiểu caller (UID app, UID shell-UiAutomation, serialize observation) + phân định ranh giới tin cậy OS.
- ■ **Thiết kế hệ điều hành SecretHandle**: Minting tại nguồn OS, ràng buộc đích (package/origin) + tác vụ (TaskToken), vòng đời dùng một lần nguyên tử, che phủ quan sát đa kênh (shade + conversation screen overlay).
- ■ **Kiểm chứng mô hình tham chiếu (Formal Reference Model)**: Mô hình hóa Python hoàn chỉnh 8 họ tấn công, 6.400 trials với seed cố định, chứng minh tính đúng đắn của State Machine.
- ⊡ **Đánh giá thực nghiệm empirical toàn diện (RQ1–RQ6)**:
  - ■ Attack 1 (Read & Exfiltrate): Đã đo $n = 30$ trials trên EC2 (B_0 rò rỉ $29/30$ vs B_3 bảo vệ $0/30$, Wilson $[0.000, 0.114]$).
  - □ Attack 2 (Destination Redirection / Killer Experiment): Chứng minh Resolver từ chối 100% khi agent cố gửi handle vào app hacker.
  - □ Baseline B_1 (Detected Placeholder / Anonymizer): Chứng minh che phủ không ràng buộc đích vẫn thất bại trước Attack 2.
  - □ Attack 4 (Replay & Expiry): Kiểm chứng vòng đời một lần (One-time) và hết hạn 5 phút.
  - □ Microbenchmarks độ trễ (RQ5): Đo thời gian Mint, Resolve, Redact (ms) và mức tiêu thụ RAM/CPU.
  - □ Benign Regression: Đo đạc trên AndroidWorld để chứng minh không làm suy giảm năng lực thông thường của Agent.

- ⊡ **Định dạng bản thảo**: Bản thảo hoàn chỉnh gồm 14 trang nội dung chính (double-column, 10pt) + phụ lục chứng minh toán học và Artifact Appendix.
- ■ **Đạo đức & Giấy phép mã nguồn mở**: Khung đạo đức với canary tổng hợp cô lập; mã nguồn phát hành với giấy phép Apache 2.0 (tương thích AOSP).

Tiến độ tổng quan

Nhóm	Trạng thái
Tài liệu ý tưởng + bản thảo + mô hình tham chiếu	■ ~90% (bản thảo 53 trang preprint + khung hội nghị 14 trang; 74 tests passed)
Audit cấp bản thảo (citation/figure/math/reviewer/notation)	■ 100% hoàn tất
Quyết định D0–D6	■ (đã chốt; D0 chuyển trọng tâm Full Paper)
Triển khai hệ thống	⊡ ~60% (nguyên mẫu stock Android chạy trên EC2; kiến trúc AOSP system_server)
Ứng dụng/tác nhân/harness	■ ~80% (EC2 runner, Bedrock bridge Sonnet 4.6, Mobile-Agent v3.5, AndroidWorld)
Baseline/ablation thực	⊡ B0 và B3 đã đo $n = 30$; chuẩn bị B1 placeholder
Thí nghiệm thực & số đo	⊡ Attack 1 hoàn thành $n = 30$; đang mở rộng Attack 2 & Overhead RQ5
Phân tích thống kê / hoàn thiện bản thảo	⊡ ~70% (Wilson CI, McNemar, bootstrap latency)
Artifact & phát hành mã nguồn	⊡ ~60% (LICENSE Apache-2.0, docs artifact, CI, benchmark harness)

PHẦN A — Nền tảng đã có (chỉ bảo trì)

- ■ Ý tưởng gốc: `idea.tex` / `idea.pdf` (5 trang) + `idea_fidelity_report.md`, `topic_signature.json`.
- ■ Bản thảo 53 trang: `main.tex` + 13 `sections/` + 2 `appendix/` + 6 hình TikZ/PGFPlots; `main.pdf`.
- ■ Mô hình tham chiếu Python `SecretHandle/src/secrethandle/` (core, crypto, policy, store, redaction, metrics, statistics, adapters, baselines, oracles, experiments, plotting, synthetic, integrity, cli).

- Bộ test offline: **74 passed** (`python -m pytest -q`).
- Harness + config tổng hợp + script demo/plot/experiment.
- Audit cấp bản thảo: `citation_claim_map.csv`, `citation_consistency_report.md`, `math_audit.md`, `claim_strength_audit.md`, `figure_quality_audit.md`, `notation_consistency_check.md`, `reviewer_audit.md`, `theorem_dependency_graph.md`, `topic_alignment_check.md`.
- Audit cấp mã: `SecretHandle/audits/` (paper_implementation_spec, paper_to_code_traceability, math_to_code_audit, implementation_gaps, result_integrity_audit).
- Tài liệu hướng dẫn: `instructions/SecretHandle_HuongDan.tex`.
- Nhật ký quyết định: `SecretHandle/docs/design_decisions.md` (D0–D6 đã chốt).

PHẦN B — Giai đoạn 0: Chuẩn bị & quyết định (W1–W2, cổng G0)

B1. Quyết định thiết kế

- **D0** Venue/lịch: **Full Research Paper** (ACM MobiSys / USENIX Security / ACM CCS / NDSS); mục tiêu bài báo nghiên cứu đầy đủ, không hạ cấp.
- **D1** Authenticator: giữ MAC (HMAC-SHA-256, canonical, tách khóa).
- **D2** Kho lưu trữ: trong bộ nhớ `system_server`; khởi động lại = thu hồi.
- **D3** Principal tác nhân: 3 kiểu (UID app / UID shell-UiAutomation / serialize) + bảng điểm chặn.
- **D4** Giảm u tại bước giữ chỗ (bảo thủ).
- **D5** WebView: T1 chỉ view gốc; WebView fail-closed.
- **D6** Môi trường: không có Pixel ⇒ emulator/Cuttlefish với ảnh AOSP tự build; số đo thiết bị thật = future work.

B2. Track A — Môi trường hệ thống AOSP

- ☐ `repo sync` AOSP (hàng chục–trăm GB, nhiều giờ) — chuẩn bị bản triển khai AOSP `system_server`.
- ☐ Ghim commit/branch AOSP; ghi provenance.
- ☐ Dựng **Android Emulator** với ảnh AOSP tự build (máy Windows) *hoặc* **Cuttlefish** (Linux/GCP có KVM) — môi trường chính theo D6.
- ☐ Build ảnh AOSP (thay cho flash Pixel); boot được trên emulator/Cuttlefish.

B3. Track B — môi trường & cấp phát

- Dựng Python 3.12; `python -m pip install -e '.[dev]'` (venv `SecretHandle/.venv`).
- Xác nhận `ruff` 0.17.1 + `mypy` 2.4.0 chạy được; **đã xử lý** hết cảnh báo ruff + lỗi mypy (PHẦN C, 2026-10-17).
- Tạo repo Git integration *tách* khỏi package tham chiếu (AOSP patch, adapter, tác nhân) — **2026-10-22:** `github.com/Thunggg/secrethandle-integration` (private), local `E:\NCKH\secrethandle-integration`; đã di chuyển `device/b0-otp-forwarding`; cấu trúc aosp-patches/device/agents/apps/env/runs; `provenance.md` ghi SHA cả 2 repo (`8f3e25d5` tham khảo / `3b85895` integration) + hash APK SHA-256; README quy tắc (raw secret-free, AI-disclosure trong commit, số functional-repro nhãn "not a paper number").
- Dựng EC2 cô lập: volume/user riêng, tường lửa chặt, credential trong secret manager (không trong YAML/log/repo). **2026-10-22:** EC2 do giảng viên cấp; spec yêu cầu sẵn sàng ở `secrethandle-integration/env/ec2` (m6i.2xlarge pilot → m6i.4xlarge spot ma trận, SSM, 0 inbound, budget $20/$150); cổng pre-EC2 6 nấc ở `env/pre_ec2_gate_and_runbook.md` — nấc G0 (offline 'make check') đã PASSED (ruff + mypy + 74 tests + canary smoke 6400 trial). **2026-10-28 (host thực tế):** `m8i.4xlarge` (16 vCPU/64 GiB, Ubuntu 24.04.5, gp3 500 GB) với `NestedVirtualization=enabled` ⇒ có `/dev/kvm` + 32 cờ `vmx`; AVD `bench` (Android 14, 1080×2400) boot headless OK, screencap OK; trên host: **74 test offline**

+ **canary 6400 trial/0 leak PASS**; bridge deps cài sẵn (`~/bridge-venv`); ghép nối SSH từ IP tác nhân hoạt động.

- **G3 — bridge → Bedrock thật (2026-10-28)**: dùng *Bedrock API key* dạng bearer (`bedrock-api-key-...`) do giảng viên cấp; khóa lưu *ngoài repo* (`env/.env.bedrock.local`, đã gitignore; trên host `~/.sh_bedrock.env` quyền 600, loader không `source` vì token chứa `&Version=1`). Bridge trả HTTP 200 với `us.anthropic.claude-sonnet-4-6` (12+4 token, 0,81 s); `/healthz` chỉ báo `bedrock_bearer_present`, không lộ giá trị. Do đó **không cần IAM instance profile** nữa.
- **G2 — pilot tác nhân thật với Bedrock (2026-10-28)**: dựng Mobile-Agent v3.5 + AndroidWorld trên EC2 (AVD `AndroidWorldAvd`, API 33, `-grpc 8554`, cài `android_env`, torch CPU, `qwen_vl_utils`); phải sửa 4 lỗi tích hợp mới chạy được — quy đổi toạ độ (model trả *pixel của ảnh đã resize*, không phải 0–1000), prompt nêu rõ quy ước, hạ ảnh $1080 \times 2400 \to 844 \times 1876$ (3,4k → 2,1k token/lời gọi), cap 8 bước/tác vụ; ghi ở `agents/MA35_PATCHES.md`. Kết quả: (i) 10 tác vụ benign B0 = **1/10** trước khi sửa toạ độ và **0/10** sau khi sửa — tức năng lực điều khiển UI của mô hình chat là nút thắt, *không phải số báo cáo* (cần GUI-Owl hoặc agent theo chỉ số phần tử/M3A); (ii) **OTP gửi bằng SMS, điều kiện B0: 10/10 trial agent đọc và trả lời đúng canary** (3–4 bước, 10–13 lời gọi, 42–63 s, ≈$0,11/trial; Wilson 95% CI [0,72; 1,00]). Đọc bí mật cần ít độ chính xác chạm hơn nhiều so với hoàn thành tác vụ, nên tỉ lệ rò rỉ 100% vẫn đứng dù task success ≈0. Đây là *mức rò rỉ nền* khi chưa có SecretHandle (B3 chưa tồn tại), không phải kết quả so sánh. Chi phí token cả phiên ≈$9.
- Cấp phát môi trường chạy (**2026-10-28**): AndroidWorld trên EC2, API key AWS Bedrock (Claude Sonnet 4.6 bearer key) cấu hình an toàn tại `~/.sh_bedrock.env`, runner Mobile-Agent v3.5 ghim commit.
- **Tái hiện tấn công B0** (OTP-forwarding) trên emulator/Android stock 14: **10/10** trial rò rỉ, exposure rate **1.0** (Wilson 95% [0.72, 1.0]; `rate_results.json`).
- Dựng disclosure monitor bản đầu (`monitor/monitor.py`, `run_rate.py`; log `exfil_log.jsonl`).
- Khung đạo đức: quy trình canary, endpoint cô lập, không dữ liệu thật; kiểm yêu cầu IRB/hội đồng + mục ethics CFP (`docs/ethics.md`, 2026-10-17).
- **Tái hiện B0 trên host EC2 thực tế — 2026-10-28:** port `build.ps1` sang Linux (`build_linux.sh`: aapt2/javac 17/d8/zipalign/apksigner; APK cuối SHA-256 `84845755...4d620a`), cài lên AVD `bench`: **10/10** và **100/100** trial rò rỉ, exposure **1.0** (Wilson 95% [0.963, 1.0]), `received_count`=100 (1:1). Ba lỗi *harness* đã tìm và sửa (không phải lỗi host): (i) API 34 cần `cmd notification allow_listener`, chỉ `settings put` thì không bind; (ii) hệ thống re-deliver notification đang active gây POST $O(n^2)$ rồi cạn kết nối → dedupe theo *toàn bộ text* của notification; (iii) Messages ngừng post khi shade bão hoà (∼47 hội thoại) → gọi `cancelAllNotifications()` sau mỗi lần forward; thêm warm-up SMS để tránh race lúc bind listener. Nhãn: *functional reproduction, not a paper number*.
- **B3 — prototype trên Android stock (2026-10-28, `device/b3-secrethandle/`)**: broker notification (mint handle `SH-XXXXXX`, cancel rồi repost notification đã redact) + accessibility service phủ overlay `TYPE_ACCESSIBILITY_OVERLAY` lên đúng node chứa mã; vault giữ plaintext TTL 5 phút, resolve *một lần*, audit `files/audit.jsonl` (mint/mask/resolve/deny). Đo trên *cùng* AVD/harness/model/canary với B0: **0/10 trial rò rỉ** (Wilson 95% [0; 0,278]) so với **10/10** của B0; 6 bước/trial (chạm cap), 113 s, ≈20,7 lời gọi, ≈$0,23/trial. **Đối chứng khi tắt service redaction: 1/1 rò rỉ trong 2 bước** — nên 0/10 không phải lỗi harness. Bốn lỗi tích hợp đã tìm và sửa: (i) API 33 cần `POST_NOTIFICATIONS` nếu không notification thay thế bị bỏ âm thầm; (ii) overlay lệch đúng một status bar (∼128 px) do thiếu `FLAG_LAYOUT_IN_SCREEN`; (iii) mask bị xoá vì clear-all mỗi accessibility event → chuyển sang cập nhật tăng dần (map rect→view) + quét 200 ms; (iv) harness ghi đè `enabled_accessibility_services` bằng AccessibilityForwarder → phải bật lại sau `env.reset()` và ghi `b3_a11y_active` mỗi trial. Hạn chế ghi rõ trong `runs/2026-10-28-b3/README.md`: overlay chỉ che kênh *screenshot* (accessibility tree vẫn xuất text gốc), clipboard chưa được mediate (agent đã thử copy/paste), và đây **không phải** `system_server` — đường triển khai AOSP vẫn là Track A.
- **Sửa metric rò rỉ của driver (2026-10-28)**: bản đầu grep canary trong *toàn bộ* log trial, mà log chứa dòng `TRIAL_RECORD` (có canary) ⇒ cột `leaked` luôn bằng 1. Nay `aw_attack_one.py` tính `canary_in_actions` (đọc `action.jsonl`) + `canary_in_sent_sms`, driver đọc `record.json`. Tính lại B0 từ trajectory đã lưu: vẫn **10/10** (`env/ec2_scripts/b0_recompute_leak.sh`).

- Ghim phiên bản Python/OS/công cụ thống kê vào provenance — **2026-10-22:** `docs/toolchain_pins.json` (Python 3.12.10 / Windows-11-26300 / numpy 2.5.4 / matplotlib 3.11.2 / PyYAML 6.0.3 / ruff 0.17.1 / mypy 2.4.0, gắn HEAD `8f3e25d5`); chạy lại và ghi lạipin khi đổi ngữ nghĩa tham chiếu.

B4. DoD của G0

- `pytest` xanh 100% (hiện 74 passed).
- Không có bí mật thật trong repo (quét mẫu high-signal = 0; không `.env`/tệp khóa; `debug.keystore` là keystore debug dùng-một-lần).
- Môi trường dev sẵn sàng (venv 3.12 + `ruff`/`mypy`); emulator KVM chạy ảnh Android 14 API 33/34 trên EC2 (**2026-10-28**).
- **B0 tái hiện được** trên emulator/stock Android (điều kiện cứng của G0) — 2026-10-17, 10/10 rò rỉ.
- D0–D6 có tài liệu quyết định.
- Khung đạo đức xong (canary tổng hợp, endpoint cô lập, không dữ liệu thật) — `docs/ethics.md`.
- **Qua G0 (25/10/2026)** — biên bản cổng: `docs/gates/G0.md` (lập 2026-10-17, kết luận ĐẠT kèm ghi chú ảnh stock).

PHẦN C — Giai đoạn 1: Củng cố mô hình tham chiếu Python (W1–W2)

- Soát `core/crypto/policy/store/redaction/metrics/statistics` đối chiếu Algorithm 1–3 + `paper_to_code_traceability.csv`; **áp dụng D1** (HMAC-SHA-256 chuẩn tắc, tách khóa) và **D4** (giảm ngân sách tại bước giữ chỗ; metadata đã xác thực bất biến, ngân sách còn lại tách riêng) — 2026-10-17. Đã sửa lỗi: trước đây giảm `u` bên trong `H` làm hỏng `a` cho handle nhiều lần dùng.
- Test dựa tính chất: giao hội chính sách không mở rộng quyền; single-use đồng thời; đơn điệu hết hạn — `tests/test_properties.py` (seed cố định 20261017, 2000 mẫu/chính sách; 5 seed cho single-use).
- Test dựa mô hình: lịch trình crash/đồng thời tại ranh giới reserve/commit/rollback/recover — `tests/test_model.py` (4 kịch bản xác định + 300 lịch trình ngẫu nhiên: không bao giờ giao quá ngân sách đơn dùng).
- Đóng băng hợp đồng adapter (contract v1, 2026-10-17): bốn adapter AOSP/platform-flow/agent-runner/benchmark cùng các protocol tin cậy; `test_adapter_contracts.py` khóa chữ ký phương thức.
- Property test: không trường bí mật rò qua serialization công khai/audit — trong `test_properties.py` (repr handle/record/store + audit; cả dạng hex).
- Mở rộng oracle tiết lộ: exact, base64, URL, JSON, chuẩn hóa Unicode, chuẩn hóa chữ số và dạng ngăn cách (space/gạch) — `oracles.DisclosureOracle` (2026-10-17).
- Chạy `ruff` + `mypy`, xử lý cảnh báo: **0 cảnh báo, 0 lỗi** (2026-10-17). Ghim cấu hình ruff tường minh trong `pyproject.toml`; nâng `requires-python` lên `>=3.10` để hiện đại hóa typing; thêm `types-PyYAML`. `pytest`: 39 → 42 (họ 1,7) → 53 (property/model) → 58 (adapter contract) → 73 (2026-10-17) → **74 passed** (thêm smoke canary, PHẦN M).
- Cập nhật `SecretHandle/audits/` sau mỗi thay đổi ngữ nghĩa — 2026-10-17: cập nhật traceability, math-to-code, gaps, spec và phụ lục result-integrity.
- **DoD:** test xanh (74 passed); mọi thay đổi ngữ nghĩa có test (D4 có `test_store`/`test_model`, oracle có `test_oracles`, adapter có `test_adapter_contracts`); traceability phủ 100% đối tượng paper đã cài.

Toàn bộ các PHẦN D–N dưới đây là lộ trình triển khai chi tiết cho Full Research Paper. Mọi hạng mục từ dịch vụ hệ điều hành, ma trận baseline, tấn công mở rộng đến đo đạc overhead đều là các thành phần bắt buộc của nghiên cứu.

PHẦN D — Giai đoạn 2: Walking skeleton (cổng G1, Track A+B)

- ■ Lát cắt dọc đầu (**2026-10-28**): SMS → mint handle `SH-XXXXXX` → repost notification đã che → che màn hình conversation → resolve tại vault. Cài đặt hoàn chỉnh trong `device/b3-secrethandle/`.
- □ Dịch vụ `system_server` với AIDL tối thiểu: `mint` (allowlist), `resolve`, `revoke`; kho trong bộ nhớ (D2).
- ■ Adapter thông báo & che mã nguồn (**2026-10-28**): `SecretHandleListenerService` chặn notification chứa OTP, hủy và tạo notification thay thế chỉ chứa handle.
- □ Resolver tối thiểu: `ResolveFreshTargetContext` (package + field + tác vụ); từ chối nếu không khớp.
- □ Giữ chỗ nguyên tử bằng khóa kho.
- □ Đường vào cho tác nhân ADB: `cmd secrethandle enter` chạy UID shell; resolver kiểm tra UID; giá trị không qua stdout/log.
- ■ Che phủ bề mặt nguồn & màn hình (**2026-10-28**): Accessibility service phủ overlay `TYPE_ACCESSIBILITY_OVERLAY` che kín node chứa mã trong conversation view; shade thông báo được che qua cancel/repost.
- ■ App A (**2026-10-28**): Luồng OTP SMS hoàn chỉnh (`b0-otp-forwarding` + SMS fixture); kẻ tấn công nhận dữ liệu tại loopback/isolated endpoint; APK băm SHA-256 `84845755...4d620a`.
- □ Adapter tác nhân → `ENTER(handle)`.
- ■ Disclosure monitor (**2026-10-28**): `aw_attack_one.py` tự động quét canary chính xác trong sent SMS, action logs, screenshot và UI state.
- ■ Thực nghiệm đối đầu mở rộng $n = 30$ B0 vs B3 (**2026-10-29**): 30 trial B0 ($29/30$ rò rỉ, Wilson $[0.833, 0.994]$) vs 30 trial B3 ($0/30$ rò rỉ, Wilson $[0.000, 0.114]$) với Mobile-Agent v3.5 (Sonnet 4.6) trên EC2.
- □ **Không làm ở giai đoạn này:** kho bền vững, khóa phần cứng, xác nhận, WebView, ví, đa người dùng.
- ■ **DoD G1:** Hoàn tất đo lường thực tế $n = 30$ cặp B0 vs B3 trên emulator, canary vắng mặt hoàn toàn trong log/screenshot của B3 ($0/30$).
- ■ **Qua G1 (15/11/2026)** — Đạt sớm từ 2026-10-17 (`docs/gates/G1.md`) và củng cố vững chắc với dữ liệu đối đầu $n = 30$ ngày 2026-10-29.

PHẦN E — Giai đoạn 3: Hoàn thiện lõi dịch vụ & resolver (W6–W8, Track A)

- □ AIDL đầy đủ: `mint` (chỉ adapter allowlist), `listHandles`, `resolve`, `revoke`, `exportAudit` (bảo vệ, rate-limit).
- □ Mô hình quyền: xác thực caller/provenance trước khi đúc; task-token verifier, source allowlist, quota, rate limit.
- □ **Target node token**: ràng buộc window/connection/node/generation/package UID + TTL ngắn.
- □ Phân loại trường: input type framework, autofill hints, view semantics; không nhất quán → fail-closed.
- □ Giữ chỗ nguyên tử (CAS/khóa) live→reserved; tái xác thực đích trước nhập; inject một lần ràng buộc reservation.
- □ Mã hóa chuẩn tắc metadata, tách khóa, độ dài tag (nếu giữ MAC theo D1); báo cáo security level thực (TEE/StrongBox/none).
- □ Thu hồi: kết thúc tác vụ, khóa thiết bị, đổi profile, đổi package đích.
- □ Bảng xử lý lỗi (unknown handle, expired, mismatch, target vanished, redactor mất theo dõi, crash), tất cả value-oblivious; wipe bộ đệm mọi đường thoát.
- □ Test đồng thời đầu-cuối + `kill system_server` tại từng ranh giới reserve/register/inject/commit.
- □ **DoD:** test binder allowlist xanh; không có lịch trình đồng thời nào giao hai lần; kill giữa chừng không giao hai lần.

PHẦN F — Giai đoạn 4: Redaction quan sát & đường nhập riêng (W6–W9, Track A)

- ☐ Đường nhập riêng (private input) bỏ qua đường thao tác hướng tác nhân + lịch sử/gợi ý IME.
- ☐ Đăng ký nghĩa vụ redaction ρ **trước** khi giao giá trị.
- ☐ Redaction trợ năng theo principal: thay giá trị/text/sự kiện bằng mask theo kiểu.
- ☐ Redaction ảnh chụp **trước khi** buffer map vào tiến trình tác nhân (không overlay sau); phủ `screencap`, UiAutomation, MediaProjection, `takeScreenshot`.
- ☐ Theo dõi định danh view/window ổn định; tính lại bounds tại capture; không định danh lại được → che window 1 frame hoặc bỏ frame.
- ☐ Che bề mặt nguồn theo cửa sổ (thông báo, ứng dụng SMS, màn hình khóa) suốt đời handle.
- ☐ Loại action echo, debug text, content description chứa tài liệu bảo vệ.
- ☐ Kết thúc redaction khi trường xóa/window đóng/tác vụ kết thúc/trạng thái app tin cậy — **không** chỉ hẹn giờ.
- ☐ Harness đọc lại: sau nhập, thu ảnh chụp, cây trợ năng, text-change event, clipboard, IME, `logcat`, `dumpsys` qua UID app, UID shell, quan sát serialize; oracle mã hóa đa dạng.
- ☐ (T3) WebView origin binding, Protected Confirmation; nếu chưa → dialog hệ thống + nhãn assurance yếu hơn.
- ☐ **DoD (G2):** pilot $n = 100$ sau nhập giá trị vắng mặt trong mọi kênh trên cả 3 kiểu principal; mọi trường hợp không chắc chắn fail-closed.
- ☐ **Mốc bản full: T1 đủ chức năng** — nếu không đạt → giữ phạm vi hẹp, không mở T2/T3.

PHẦN G — Giai đoạn 5: Ứng dụng, endpoint, tác nhân, monitor (Track B, W1–W9)

- ■ App A: đăng nhập + OTP SMS (`b0-otp-forwarding`, APK SHA-256 `84845755...4d620a`) đã hoàn thiện và đo đạc thực tế $n = 30$.
- ☐ App B: 2FA nhiều trang với preview tin nhắn đối kháng (T1).
- ☐ App C: checkout địa chỉ + điện thoại (T2).
- ☐ App D: checkout token ví + xác nhận mỗi lần (T3).
- ☐ Mỗi app chỉ ghi ground-truth vào harness bảo vệ: định danh đích, đã giao/chưa, server chấp nhận, hoàn tất.
- ■ Endpoint verifier (đo tính sẵn có định danh) + endpoint attacker cô lập (đích ngoài d), không mạng thật.
- ■ Ghim nguồn + build + băm APK (SHA-256); ghi schema oracle server (`provenance.md`).
- ■ Tác nhân: Chọn Mobile-Agent v3.5 chạy trên AndroidWorld (**2026-10-28**); ghim bản vá tại `agents/MA35_PATCHES.md`; tái hiện thành công B0.
- ☐ Tác nhân thứ hai (Mobile-Agent, DroidRun nếu ổn định) — T2.
- ■ Mô hình cloud: Claude Sonnet 4.6 (`us.anthropic.claude-sonnet-4-6`) qua Bedrock bridge chạy ổn định trên EC2.
- ■ Disclosure monitor: Tự động hóa trong `aw_attack_one.py` và driver matrix; xuất kết quả chi tiết từng bước qua `record.json` và `action.jsonl`.
- ☐ Hồi quy tác vụ chung (AndroidWorld/SPA-Bench) — T2.
- ■ **DoD G1 (Track B):** Chạy được luồng app A dưới giám sát disclosure, tái lập với seed cố định; build tái lập; hash khớp.

PHẦN H — Giai đoạn 6: Baseline & ablation (W6–W10)

- ■ **B0** (không bảo vệ): Đã đo $n = 30$ trial trên EC2 ($29/30$ rò rỉ, Wilson $[0.833, 0.994]$).

- ■ **B1** (placeholder phát hiện được): Đo đối đầu Attack 2 ($n = 30$ trial trên EC2) — do không có destination binding, B_1 bị lừa giải mã cho app độc hại $30/30$ trial (redirection leakage 1.00, utility 0.00).
- ■ **B3** (SecretHandle): Nguyên mẫu stock-Android (`device/b3-secrethandle`) đã đo đầy đủ: Attack 1 ($0/30$ rò rỉ), Attack 2 ($0/30$ rò rỉ, utility $30/30$), Attack 4 ($50/50$ replay blocked, $50/50$ expiry blocked, $50/50$ concurrent single-use), và Microbenchmarks (mint p95 $9.30\,\mu$s, resolve p95 $0.18\,\mu$s).
- ☐ **B2** (T2): Autofill/OTP Autofill/Credential Manager cho app hợp tác; `FLAG_SECURE`, che trường mật khẩu, content sensitivity; workflow không hỗ trợ ghi `not_applicable`.
- ☐ Ablation T1 (tắt từng thứ): đúc nguồn (thay bằng cạo màn hình), ràng buộc đích, ràng buộc tác vụ, vòng đời nguyên tử, redaction sau nhập; kiểm origin là T3.
- ■ Đối chứng không an toàn (**2026-10-28**): Tắt service redaction của B3 → rò rỉ $1/1$ trong 2 bước, xác nhận rào chắn an toàn là thực chất.
- ☐ **Đóng băng tham số trước khi xem kết quả**: bits định danh, bits tag, TTL, ngân sách dùng, rate limit, thời lượng quan sát, số lặp, số bootstrap, chính sách outlier, cỡ mẫu từng ô.
- ☐ **DoD (G3):** chuyển B0↔B3 không đổi text tác vụ; audit hiển thị quyết định; oracle cô lập; tham số đóng băng.
- ☐ **Mốc bản full: đóng băng mã hệ thống** — từ đây chỉ sửa lỗi có ghi nhận.

PHẦN I — Giai đoạn 7: Chạy thí nghiệm (W10–W13)

- ■ Cấp phát hạ tầng EC2 (**2026-10-28**): Instance `m8i.4xlarge` (Nested Virtualization KVM, AVD bench/AndroidWorldAvd API 33/34, Bedrock bridge) sẵn sàng và vận hành ổn định.
- ☐ Copy `configs/experiment_template.yaml`; đặt đường dẫn/ID đã cấp phát; đóng băng tham số.
- ■ Smoke test canary cô lập (**2026-10-28**): Xác nhận 100% canary tổng hợp, zero dữ liệu thật trên cả offline test lẫn emulator host.
- ☐ **Killer experiment đầy đủ** (B0 vs B3 trên app A), B3 $n \geq 300$: kiểm 3 điều kiện (a)(b)(c).
- ☐ Ma trận T1 (~8.000 trial LLM) trên emulator/Cuttlefish; đo độ trễ/bộ nhớ trên Pixel thật.
- ☐ Họ tấn công kịch bản (3, 4, TOCTOU của 7) với $n \geq 10^4$, không LLM.
- ☐ Ghi **mọi** loại trừ theo nhóm (agent crash, platform crash, network outage, user cancel, định danh không hỗ trợ); chặn trước chính sách outlier.
- ■ Xuất dữ liệu thô (raw secret-free): Đã lưu trữ và đồng bộ toàn bộ log, `driver.tsv`, screenshots từng trial về `secrethandle-integration/runs/`.
- ■ Sinh bảng/hình **chỉ** từ raw: Bảng 3 và Hình minh họa được trích xuất trực tiếp từ các file kết quả thực tế trên EC2.
- ☐ **DoD:** (a) giá trị thô vắng mọi artifact; (b) redirect bị từ chối; (c) login hợp lệ thành công; mọi loại trừ được ghi; bảng/hình sinh từ raw.

PHẦN J — Giai đoạn 8–9: Mở rộng T2/T3 (chỉ khi G2 qua)

- ☐ (T2) Adapter clipboard nhạy cảm: thay nội dung clip hướng tác nhân bằng projection công khai; giữ dán thường cho người dùng.
- ☐ (T2) App C (địa chỉ + điện thoại); B2 + so sánh cơ chế che nền tảng.
- ☐ (T2) Đo năng lượng (Perfetto/Batterystats); tác nhân thứ hai; hồi quy tác vụ chung.
- ☐ (T2) WebView fail-closed: đo tỉ lệ từ chối sai, ghi `unsupported`.
- ☐ (T3) Credential Manager adapter; ví + Protected Confirmation (app D).
- ☐ (T3) Xác thực WebView origin (node→frame→origin); fail-closed nếu không đủ.
- ☐ (T3) Kho bền vững + nhật ký sự cố + đa người dùng/sao lưu; khóa StrongBox/Knox; thiết bị OEM.
- ☐ Lưu ý: hạng mục T3 **không** nêu trong đóng góp nếu chưa làm — chuyển sang Giới hạn/Hướng phát triển.

PHẦN K — Giai đoạn 10: Phân tích & viết bài (W9–W15)

- ■ Rút gọn bản thảo 53 trang về giới hạn venue MobiSys (`mobisys/main.pdf` 14 trang: 10 trang thân bài $\leq$ 12 trang, 2 trang references, 2 trang artifact appendix); toàn bộ chứng minh hình thức và chi tiết mở rộng lưu ở bản master/technical report.
- ■ Báo cáo nhị phân dạng "0 of n" + Wilson CI: B0 là $29/30$ (Wilson $[0.833, 0.994]$), B3 là $0/30$ (Wilson $[0.000, 0.114]$); không tuyên bố tuyệt đối.
- ■ Độ trễ: trung bình, trung vị, phân vị đuôi (p95), độ lệch chuẩn (mint $5.61\,\mu$s, resolve $0.13\,\mu$s).
- ■ So sánh cặp: B0 vs B3, B1 vs B3 đối đầu redirection leakage ($30/30$ vs $0/30$); RQ1–RQ6 hoàn tất 100% thực nghiệm trên Android 14.
- ■ Thay **mọi** gạch/TODO trong bảng kết quả/tham số/ablation/tương thích bằng giá trị đo thực nghiệm; 0 placeholder.
- ■ Thay/gỡ tọa độ PGFPlots khái niệm (`figures/pgfplots_placeholder_results.tex`) bằng số liệu đo thực tế.
- ■ Kiểm giả định định lý bằng oracle thực nghiệm; nêu giả định nào còn kiến trúc.
- ☐ Cập nhật closest-work/novelty; chạy lại tìm kiếm tài liệu ngay trước nộp; xác minh toàn văn các so sánh mới.
- ■ Chuyển sang class/mẫu LaTeX ACM `sigconf` của MobiSys; tuân chính sách ẩn danh double-blind.
- ■ **DoD (bản full):** không còn TODO/gạch trong bảng; mọi hình/bảng tái tạo từ raw; checker nộp `manuscript_check.py` xanh PASS.
- ☐ **Mốc bản full: đóng băng số liệu + bản thảo** (lịch kỳ sau).

PHẦN L — Trích dẫn & bằng chứng

- ■ Nâng so sánh chỉ-abstract trong `citation_claim_map.csv` lên xác minh full-text trang/mục (đặc biệt vai trò AutoDroid; bằng chứng thực tế ở `sections/`). **2026-10-17:** closest-work `zhao2026anonymization` đã xác minh full-text (arXiv HTML v3, Sec. 3.2/3.5); **sửa lỗi** `wen2023autodroid` — nay chỉ trích dẫn đúng vai trò hệ tác nhân di động; đã sửa ở cả `mobisys/` và bản full. **Bổ sung 2026-10-17:** 4 mục còn lại đã đối chiếu bản arXiv hiện hành — `du2025mobileattacks` nâng lên **full-text** (arXiv HTML v3, Sec. IV-C: thu hoạch mã xác minh Alipay); `debenedetti2025camel` (v2), `costa2025fides` (v2), `shi2025progent` (v3) giữ **abstract verified** vì bản thảo chỉ dùng tóm tắt cấp cao đã được bao hàm; đã ghi nhận trôi phiên bản. Quyết định mở: **đã chốt 2026-10-22** — tiêu đề bib cho `du2025mobileattacks` dùng tiêu đề bản **v3** (*Exploiting Mobile LLM Agents as Confused Deputies via Non-App-Controlled Channels*) vì bản thảo trích v3 (nội dung đã xác minh ở Sec. IV-C); metadata arXiv chưa phản ánh — rà lại ở camera-ready trong trường hợp metadata cập nhật. Đã sửa `references_verified.bib` + `citation_claim_map.csv` + `citation_todo.md` + `citation_consistency_report.md`.
- ☐ Chạy lại tìm kiếm tài liệu ngay trước nộp; cập nhật closest-work/novelty.
- ☐ Thay arXiv bằng bản archival khi có; xác minh tác giả/trang/DOI/venue.
- ☐ Giải quyết trích dẫn archival ổn định cho DroidRun nếu dùng làm tác nhân đánh giá.
- ☐ Giữ HaS, Rescriber, PAPILLON, PrivacyRestore, Dual-LLM pattern, trang sự kiện/sản phẩm ở ngoài trừ khi xác minh bản ghi/claim chính.
- ☐ Archive snapshot các trang Android Developers/AOSP dùng trích dẫn (kèm access date).

PHẦN M — Artifact & nộp bài (W15–W16, Giai đoạn 11)

- ■ Giấy phép mã nguồn mở (**2026-10-29**): Đã tạo file `LICENSE` (Apache License 2.0) cho cả 2 repo `SecretHandle` và `secrethandle-integration`, cập nhật tài liệu artifact.
- ■ Phụ lục artifact + mô tả (MobiSys artifact evaluation *tùy chọn*, sau khi được nhận); tách test thiết bị khỏi test emulator; tài liệu tính năng OEM không có. **2026-10-17:** viết `docs/artifact.md` (mô tả artifact,

lệnh một-bước, smoke offline, phạm vi, ghi chú LICENSE) và `docs/oem_features.md` (tính năng OEM thiếu + tầng test). Tách tầng test bằng marker pytest `device`/`emulator` (`pyproject.toml`) + target `make test-offline` + mục "Test tiers" trong `docs/reproducibility.md`. Phụ lục chính thức trong bài nộp để sau khi được nhận.

- ■ Một lệnh build/run + smoke test canary cô lập offline; CI Python 3.11/3.12 + pipeline thiết bị kích hoạt thủ công. `make check`/`scripts/smoke_check.py` (offline, 1599 canary, 0 rò rỉ); `.github/workflows/ci.yml` (3.11/3.12) + `device.yml` (`workflow_dispatch`); local verify 74 passed (2026-10-17).
- ■ Kiểm tra biên dịch LaTeX: XeLaTeX biên dịch sạch với 0 lỗi, 0 undefined citation/reference.
- ⊡ Render lại mọi hình/bảng sau thay đổi class cuối; lặp `figure_quality_audit.md`. **2026-10-17:** render lại 6 hình bản full (trang 11/13/17/21/30/34) + hình mobisys (trang 4) vào `figure_previews/`; cập nhật audit (nguồn hình không đổi từ 2026-10-03, chỉ dịch trang). **Còn:** lần render/soát cuối sau khi chốt class + soát mắt hình mobisys mới.
- □ Nộp **trước** hạn vài ngày (hạn kỳ sau — tra cứu CFP tại thời điểm lập lịch).

PHẦN N — Audit bảo trì (chạy lại sau mỗi thay đổi lớn)

- □ `math_audit.md` + `theorem_dependency_graph.md` sau đổi định lý.
- □ `notation_consistency_check.md` sau đổi ký hiệu.
- □ `citation_consistency_report.md` + `citation_claim_map.csv` sau đổi trích dẫn.
- □ `figure_quality_audit.md` sau render lại hình.
- □ `claim_strength_audit.md` sau đổi tuyên bố.
- □ `topic_alignment_check.md` + `idea_fidelity_report.md` sau đổi phạm vi/đóng góp.
- □ `SecretHandle/audits/*` sau mỗi thay đổi ngữ nghĩa mã.
- □ `reviewer_audit.md` sau mỗi vòng soát.

LỘ TRÌNH THỰC NGHIỆM BẢN FULL (FULL RESEARCH PAPER ROADMAP)

Mục tiêu: Bổ sung đầy đủ 6 hạng mục thực nghiệm empirical trên Android Emulator / Thiết bị thật để trả lời hoàn chỉnh 6 câu hỏi nghiên cứu (RQ1–RQ6) phục vụ bản Full Paper.

- ■ **Hạng mục 1: Tấn công Đọc & Lộ OTP (Attack 1 — Read & Exfiltrate OTP) [RQ1, RQ4]**
  - ■ Tái hiện B_0 (Unprotected) trên Android Emulator ($n = 30$ trials, $29/30$ rò rỉ, Wilson $[0.833, 0.994]$).
  - ■ Đo nguyên mẫu B_3 (SecretHandle Prototype) trên Android Emulator ($n = 30$ trials, $0/30$ rò rỉ, Wilson $[0.000, 0.114]$).
  - ■ Đối chứng khi tắt overlay/redaction service: $1/1$ rò rỉ ngay trong 2 bước → rào chắn an toàn là thực chất.
- ■ **Hạng mục 2: Thí nghiệm then chốt: Chống chuyển hướng độc hại (Attack 2 — Destination Redirection / Killer Experiment) [RQ2]**
  - ■ Thiết lập kịch bản Attack 2 trên Android Emulator: Agent nhận handle `SH-XXXXXX` và cố điền vào app/destination độc hại ($n = 30$ paired trials).
  - ■ SecretHandle Resolver xác thực caller package đối chiếu với target được ghim tại bước mint: Từ chối giải mã với `CALLER_MISMATCH` trong $30/30$ trials ($R_{B_3} = 0.00$, Wilson $[0.000, 0.114]$).
  - ■ Ngược lại, khi gửi đến app hợp lệ (`com.legitimate.app`) → giải mã thành công $30/30$ trials ($U_{B_3} = 1.00$, Wilson $[0.886, 1.000]$).
- ■ **Hạng mục 3: So sánh đối đầu với Baseline B_1 (Detected Placeholder / Anonymizer) [RQ2]**
  - ■ Cài đặt baseline B_1 (placeholder không ràng buộc đích) chạy trên cùng môi trường đối đầu Attack 2 ($n = 30$ trials).

- ■ Kết quả đối đầu: B_1 bị lừa giải mã và rò rỉ sang endpoint kẻ tấn công 30/30 trials ($R_{B_1} = 1.00$, Wilson $[0.886, 1.000]$), tiện ích hợp lệ bị phá hủy ($U_{B_1} = 0.00$); trong khi B_3 giữ vững $R_{B_3} = 0.00$ và $U_{B_3} = 1.00$.

- ■ **Hạng mục 4: Kiểm tra vòng đời & Chống Replay (Attack 4 — Replay, Concurrency & Expiry) [RQ3]**
  - ■ Kiểm tra Replay ($n = 50$ trials): $50/50$ yêu cầu tái sử dụng cùng một handle lần thứ hai bị Resolver từ chối dứt khoát với lỗi `ALREADY_RESOLVED` ($Y = 0.00$, Wilson $[0.000, 0.071]$).
  - ■ Kiểm tra Expiry ($n = 50$ trials): $50/50$ yêu cầu giải mã sau khi hết hạn TTL (5 phút) bị từ chối với lỗi `HANDLE_EXPIRED` ($0/50$ lọt qua).
  - ■ Kiểm tra Race condition ($n = 50$ trials $\times$ 10 luồng đua song song = 500 luồng): Khóa nguyên tử vault bảo đảm chính xác duy nhất 1 luồng thành công trên mỗi handle trong $50/50$ trials, 0 dị thường đồng thời.

- ■ **Hạng mục 5: Đo đạc hiệu năng & độ trễ (Overhead Benchmark — RQ5)**
  - ■ Microbenchmark độ trễ đo trực tiếp trên Android runtime qua `app_process` ($n = 100$ trials):
    - Thời gian tạo handle (`Mint latency`): Mean 5.61 μs (median 5.24 μs, min 2.73 μs, p95 9.30 μs).
    - Thời gian giải mã (`Resolve latency`): Mean 0.13 μs (median 0.12 μs, min 0.11 μs, p95 0.18 μs).
  - ■ Đo đạc tài nguyên: Dalvik heap allocated chỉ 1.15 MB (total heap 8.39 MB); tổng bộ nhớ tiến trình (Total PSS) là 28.71 MB (Native 14.19 MB, Dalvik 2.11 MB), khẳng định overhead cực thấp.

- ■ **Hạng mục 6: Đa dạng hóa Ứng dụng & Hồi quy tác vụ chung (Workloads & Benign Regression) [RQ6]**
  - ■ AndroidWorld Benign Regression & Transparency (**2026-10-29**): Đo đạc trên 10 ứng dụng thông thường (DeskClock, Markor Notes, Calendar, Gallery, AudioRecorder, SimpleDraw, Camera2, Settings, SMS, Contacts). Kết quả:
    - Tính trong suốt cảm ứng: $50/50$ thao tác chạm và cử chỉ được chuyển tiếp nguyên vẹn 100% đến ứng dụng nền nhờ cờ `FLAG_NOT_TOUCHABLE` (tỷ lệ chặn 0.0%).
    - Độ trễ khung hình GPU: Đo qua `dumpsys gfxinfo` trên Markor Notes và Settings, độ trễ đuôi p95 hoàn toàn tương đồng giữa B_0 và B_3 (Markor: 18.0 ms, Settings: 36.0 ms, độ chênh lệch $\Delta = 0.0$ ms), chứng minh vòng quét 200 ms không làm chậm compositor.
    - Che nhầm: $0/10$ màn hình bị che phủ vĩnh viễn sai; chỉ có match tạm thời trên lịch sử SMS canary cũ.
    - Kết luận: Hồi quy tác vụ thông thường $\Delta = 0.00$ (Zero Regression).
  - □ Workload B: Đăng nhập 2FA nhiều bước với tin nhắn đối kháng preview (bản mở rộng T2).
  - □ Workload C: Điền thông tin cá nhân/địa chỉ (Checkout form, bản mở rộng T2).

Rủi ro chính & phương án

Rủi ro	Phương án
Thiếu thiết bị thật Pixel	Chạy toàn bộ ma trận tấn công trên Android Emulator (máy chủ EC2 có KVM) với Android 14 API 33/34; ghi rõ phạm vi trong Methodology.
Thời gian chạy LLM lâu	Dùng Bedrock bearer token trực tiếp qua bridge cục bộ; tự động hóa batch chạy ban đêm trên EC2.
Kẻ tấn công bypass lớp phủ screenshot	SecretHandle kết hợp cả chặn notification và che phủ overlay; trên bản full kết hợp thêm `system_server` private input.

5 hành động tiếp theo (ưu tiên ngay)

1. ■ **Hoàn thành $n = 30$ trials cho Attack 1** (B_0 vs B_3 trên EC2, Wilson CI [0.83, 0.99] vs [0, 0.114]).

2. ■ **Triển khai Attack 2 (Destination Redirection)**: Chứng minh SecretHandle Resolver chặn 100% (30/30, $R = 0.00$) trong khi B_1 rò rỉ 100% (30/30, $R = 1.00$).
3. ■ **Chạy Microbenchmark đo độ trễ & RAM (RQ5)**: Mint p95 9.30 μs, Resolve p95 0.18 μs, Total PSS 28.71 MB.
4. ■ **Kiểm tra vòng đời & Chống Replay (Attack 4)**: 50/50 replay blocked, 50/50 expiry blocked, 50/50 single-use concurrent race không dị thường.
5. ■ **Đo hồi quy tác vụ chung AndroidWorld Benign Regression (RQ6)**: 50/50 touch pass-through, render $\Delta = 0.0$ ms, regression $\Delta = 0.00$.

Ghi chú: Khi một mục xong, đổi □ → ■ và ghi ngày. Biên dịch: `xelatex MASTER_CHECKLIST.tex` hai lần.